

Applying CNN on MNIST Dataset

April 25, 2025

Question: Applying CNN with Three Convolutional Layers on MNIST

You are tasked with applying a Convolutional Neural Network (CNN) to the MNIST dataset, which consists of grayscale images of handwritten digits (0–9), each of size 28×28 pixels. Follow the steps below to build a CNN with three convolutional layers from scratch using a deep learning framework of your choice (e.g., TensorFlow/Keras or PyTorch).

Step 1. Load and preprocess the MNIST dataset:

- Load the dataset and split it into training and test sets.
- Normalize the image pixel values to the range $[0, 1]$.
- Reshape the data to include a channel dimension.

Step 2. Define the CNN architecture with three convolutional layers:

- First convolutional layer: 32 filters, 3×3 kernel, ReLU activation.
- Second convolutional layer: 64 filters, 3×3 kernel, ReLU activation.
- Third convolutional layer: 128 filters, 3×3 kernel, ReLU activation.
- Add MaxPooling layers after appropriate convolutional layers to reduce dimensionality.
- Flatten the output and add a Dense layer with 128 units and ReLU activation.
- Add a final Dense layer with 10 units and softmax activation for classification.

Step 3. Compile the model:

- Use `categorical_crossentropy` loss.
- Choose an appropriate optimizer (e.g., Adam).
- Track accuracy as a metric.

Step 4. Train the model:

- Fit the model on the training data.
- Use validation split to monitor performance.
- Train for a sufficient number of epochs (e.g., 10–20).

Step 5. Evaluate the model:

- Evaluate the trained model on the test set.
- Report the final accuracy and loss.

Step 6. Visualize the results:

- Plot training and validation accuracy and loss over epochs.
- Display a few test images along with their predicted and actual labels.
- What patterns do the filters seem to detect?
- Highlight any misclassifications.